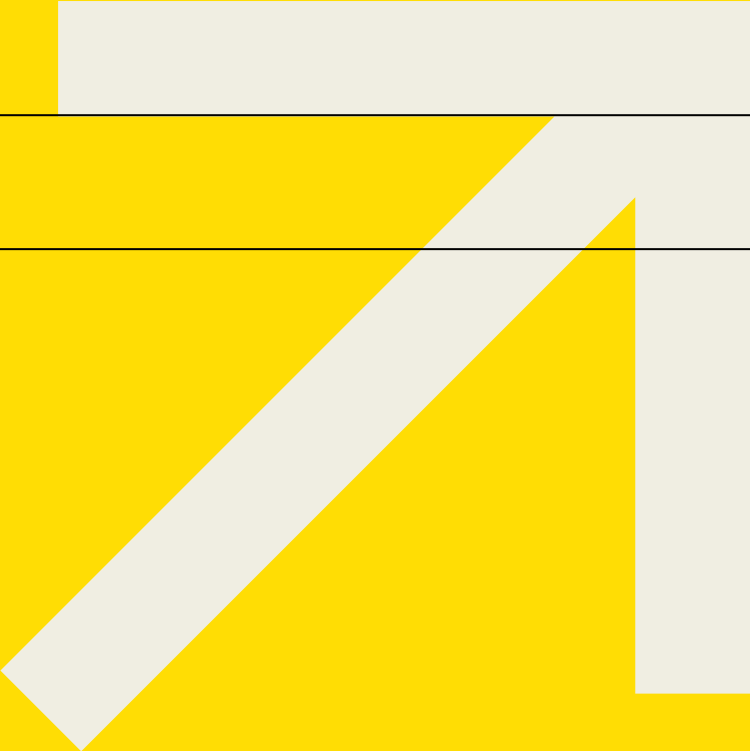


Credit risk segmentation

An abstract graphic design featuring several white geometric shapes on a yellow background. There is a horizontal rectangle at the top right, a vertical rectangle on the far right, and a large diagonal rectangle that spans from the middle left towards the bottom right.

Predicting probability of default
from a statistical , machine
learning, and deep learning
perspective.

01

Introduction & problem set up

02

Data preprocessing

03

Model building & Scorecard

04

Conclusion & future work

Project phase: Intro & Motivation



Motivation

In finance, risk often means the chance of financial loss. In banking, loan risk is mainly tied to whether a borrower defaults. Once a borrower defaults, we receive monetary loss, which is bad for business



Goal

Build a model that estimates a borrower's current probability of default using the most important risk factors if possible. Make the model practical to be used by stakeholders with non-technical backgrounds.



Data

This project used the full Lending Club loan dataset, containing about 2 million loans and 142 borrower and loan-related variables. Dates ranging from 2014-2018.



Project phase:

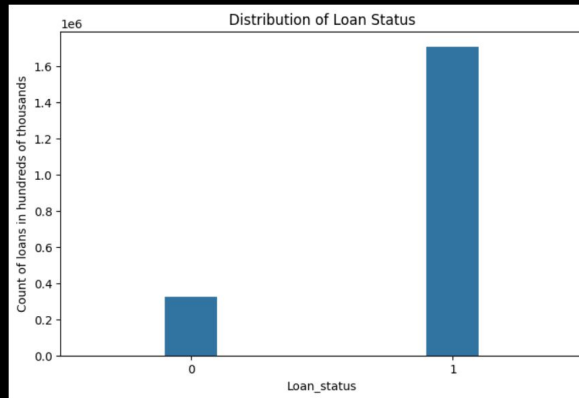
Dependent variable

y	
1	1704881
0	325071

Creation of DV

The original target variable, loan status, had 7 categories. We grouped these statuses into two classes: default risk and non-default risk, turning the problem into a binary classification task.

Good-standing loans were about 5.2 times more common than default-risk loans, showing a clear class imbalance.



```
#Create the loan mapping for the dependent variable
loan_mapping_label = {
    "Fully Paid":1, #Loan was repaid in full
    "Current":1, #borrower is currently making payments
    "Late (31-120 days)":0, #Payment is overdue
    "Charged Off":0, #lender considers the loan unlikely to be collected
    "In Grace Period": 1, #Slightly overdue payment but still within allowed extra time
    "Late (16-30 days)":0, #Payment is overdue
    "Default":0 #loan is officially failed to be repaid
}
```

Project phase:

Pipeline creation



Transformers were fit only on the training data, then applied to the validation and test sets. This prevents data leakage and keeps model evaluation fair.



Custom transformer functions were created to clean the data, engineer features, and preprocess variables. Separate pipelines were used for numerical and categorical features.

```
categorical_pipeline = Pipeline([
    ("drop_missing_cols", Missing_val_droppper(threshold=50.0)),
    ("imputer", SimpleImputer(strategy="constant", fill_value="missing", add_indicator=True)),
    ("cat_selector", CategoricalFeatureSelector(threshold=0.80)),
    ("onehot", OneHotEncoder(drop="first", handle_unknown="ignore", sparse_output=False))
])
```

```
numerical_pipeline = Pipeline([
    ("drop_missing_cols", Missing_val_droppper(threshold=50.0)),
    ("imputer", SimpleImputer(strategy="median", add_indicator=True)),
    ("outlier", OutlierFlaggerV2()),
    ("scaler", RobustScaler())
])
```

```
pipe = Pipeline([
    ("preprocess", preprocessing),
    ("corr_selector", CorrelationFeatureSelector(threshold=0.80)),
    ("tree_selector", SelectFromModel(
        RandomForestClassifier(n_estimators=300, random_state=42),
        threshold="mean"
    )),
    ("sampler", "passthrough"),
    ("model", LogisticRegression(max_iter=1000))
])
```

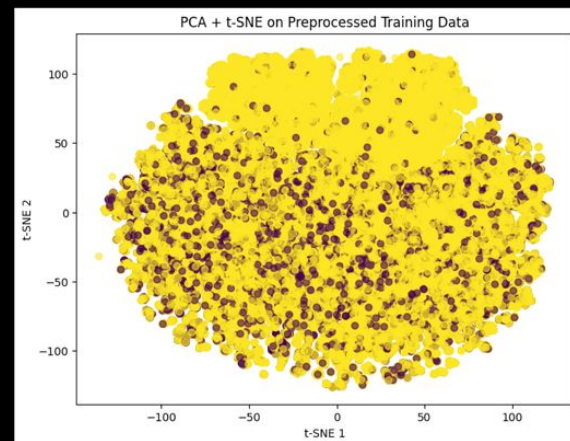
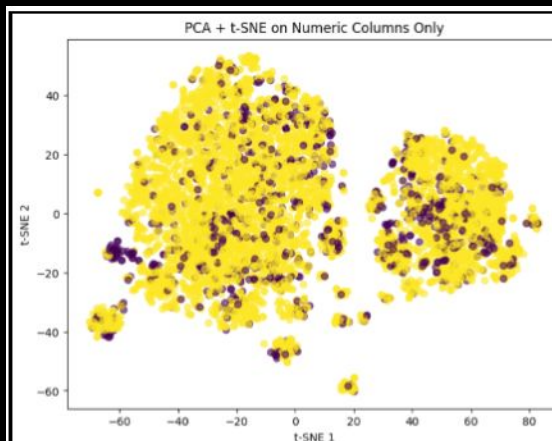
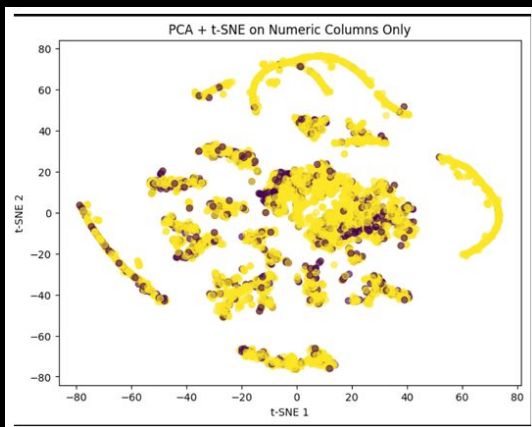
```
fe = Feature_engineering_step(date_cols=date_cols)
```

```
X_train_fe = fe.fit_transform(X_train)
```

```
X_val_fe = fe.transform(X_val)
```

```
X_test_fe = fe.transform(X_test)
```

PCA & t-NSE



Project phase: Pipeline Archetypes



To address class imbalance, three modeling pipeline types were compared: adjusting model class weights, applying sampling methods, and combining both approaches.

```
"sampler": [  
  # Undersampling  
  RandomUnderSampler(sampling_strategy=0.30, random_state=42),  
  RandomUnderSampler(sampling_strategy=0.40, random_state=42),  
  RandomUnderSampler(sampling_strategy=0.50, random_state=42),  
  RandomUnderSampler(sampling_strategy=0.70, random_state=42),  
  
  # Oversampling  
  RandomOverSampler(sampling_strategy=0.25, random_state=42),  
  RandomOverSampler(sampling_strategy=0.35, random_state=42),  
  RandomOverSampler(sampling_strategy=0.50, random_state=42),  
  
  # SMOTE  
  SMOTE(sampling_strategy=0.25, k_neighbors=3, random_state=42),  
  SMOTE(sampling_strategy=0.35, k_neighbors=3, random_state=42),  
  SMOTE(sampling_strategy=0.50, k_neighbors=3, random_state=42),  
  
  SMOTE(sampling_strategy=0.25, k_neighbors=5, random_state=42),  
  SMOTE(sampling_strategy=0.35, k_neighbors=5, random_state=42),  
  SMOTE(sampling_strategy=0.50, k_neighbors=5, random_state=42),  
  
  SMOTE(sampling_strategy=0.70, k_neighbors=5, random_state=42),
```

```
XGB_param_no_sampler = [  
  
  # XGBoost  
  {  
    "preprocess__num_drop_missing_cols__threshold": [50, 60, 70, 80, 90],  
    "preprocess__cat_drop_missing_cols__threshold": [50, 60, 70, 80, 90],  
  
    "sampler": ["passthrough"],  
  
    "model": [  
      XGBClassifier(  
        random_state=42,  
        eval_metric="logloss"  
      )  
    ],  
  
    "model__n_estimators": [100, 300, 500, 800],  
    "model__learning_rate": [0.01, 0.05, 0.1],  
    "model__max_depth": [3, 4, 5],  
    "model__subsample": [0.6, 0.8, 1.0],  
    "model__colsample_bytree": [0.6, 0.8, 1.0],  
    "model__booster": ["gbtree"], # "dart"  
  
    "model__scale_pos_weight": [scale_pos_weight, 5, 10, 20],  
  
    "model__reg_alpha": [0, 0.01, 0.1, 1],  
    "model__reg_lambda": [1, 5, 10]  
  ]  
]
```



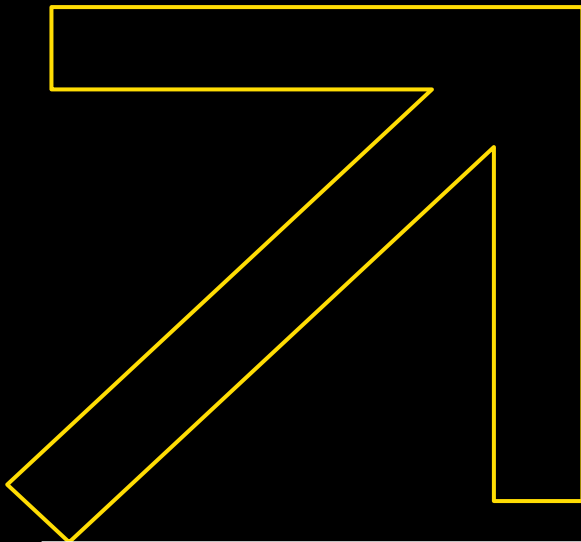
The models that were used were Xgboost classifier, random forest, Logistic regression, and Neural Networks.



Data was split into training/validation/test. We chose the model/ tuned hyperparameters in validation, and trained the final models on the test.



Hyperparameters were tuned using RandomizedSearchCV with 10 random combinations and 3-fold cross-validation to keep model selection consistent.



Project phase:

Model training & Selection

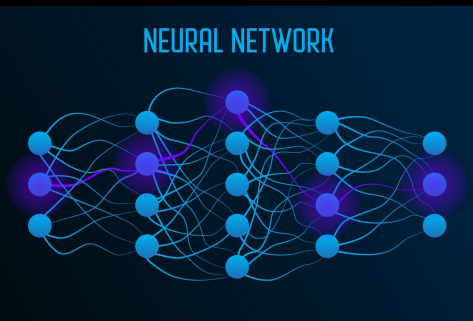
TABLE I
MODEL PERFORMANCE COMPARISON WITH RESPECT TO CLASS 0 IN
VALIDATION

Model	ROC-AUC	Precision	Recall	F1-score
Logistic Regression	0.75	0.29	0.60	0.40
Random Forest	0.745	0.29	0.61	0.39
XGBoost	0.75	0.30	0.58	0.40
Neural Network	0.79	0.43	0.49	0.45

TABLE II
MODEL PERFORMANCE COMPARISON WITH RESPECT TO CLASS 0 IN TEST
SET

Model	ROC-AUC	Precision	Recall	F1-score
Logistic Regression	0.75	0.29	0.60	0.39
XGBoost	0.75	0.30	0.59	0.40
Neural Network	0.79	0.38	0.55	0.45

NN Architecture



```
class MLPClassifier_largeNN(nn.Module):
    def __init__(self, dropout=0.30):
        super().__init__()
        #start with a dense layer to capture info
        self.network = nn.Sequential([
            nn.LazyLinear(256),

            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(p=dropout),

            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Dropout(p=dropout),

            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Dropout(p=dropout),

            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Dropout(p=dropout),

            nn.Linear(16, 2)
        ])

    def forward(self, x):
        logits = self.network(x)
        return logits
```

```
net = NeuralNetClassifier(
    module=MLPClassifier_largeNN,
    module_dropout=0.3,

    criterion=nn.CrossEntropyLoss,
    criterion_weight=class_weights,

    optimizer=torch.optim.Adam,
    optimizer_lr=0.001,
    optimizer_weight_decay=0.001,

    max_epochs=100,
    batch_size=256,
    iterator_train_shuffle=True,
    device=device,
    verbose=1,
    callbacks=[
        EarlyStopping(
            monitor="valid_loss",
            patience=10,
            threshold=0.0001,
            threshold_mode="rel",
            lower_is_better=True
        )
    ]
)
```

```
param_dist_nn = {
    # learning rate
    "model_optimizer_lr": loguniform(1e-4, 1e-2),

    # regularization
    "model_optimizer_weight_decay": loguniform(1e-5, 1e-1),

    # dropout: tries values from 0.10 to 0.50
    "model_module_dropout": uniform(0.1, 0.4),

    # batch size
    "model_batch_size": [128, 256, 512],

    # optimizer
    "model_optimizer": [
        torch.optim.Adam,
        torch.optim.AdamW
    ],

    # class imbalance weights
    # order is [class_0_weight, class_1_weight]
    "model_criterion_weight": [
        torch.tensor([1.0, 1.0], dtype=torch.float32).to(device),
        torch.tensor([2.0, 1.0], dtype=torch.float32).to(device),
        torch.tensor([3.0, 1.0], dtype=torch.float32).to(device),
        torch.tensor([5.0, 1.0], dtype=torch.float32).to(device),
        torch.tensor([10.0, 1.0], dtype=torch.float32).to(device),
    ]
}
```

```
nn_pipe = Pipeline(steps=[
    ("preprocess", preprocessing),
    ("to_float32", FunctionTransformer(lambda X: X.astype(np.float32), validate=False)),
    ("model", net)
])
```

Project phase:

Selected models

Model outputs were translated into practical scorecards. Logistic Regression used the top 10 features by absolute coefficient value, while XGBoost and the Neural Network used binned predicted probabilities based on the best F1 threshold.

Best XGBoost Sampler Model

```
-----  
Model file: best_XGB_sampler_model.pkl  
Model type: XGBoost classifier with sampling pipeline  
n_estimators = 300  
learning_rate = 0.05  
max_depth = 5  
subsample = 0.8  
colsample_bytree = 1.0  
reg_alpha = 0  
reg_lambda = 10  
booster = gbtrees
```

Best L2-Weighted Logistic Regression Model

```
-----  
Model file: lr_l2_weights_model.pkl  
Model type: L2-weighted Logistic Regression  
sampler = passthrough  
numerical missing-value threshold = 50  
categorical missing-value threshold = 90  
penalty = l2  
solver = saga  
C = 0.01  
class_weight = {0: 5, 1: 1}  
tol = 0.001
```

Best params for large NN:

```
'model__batch_size': 256,  
'model__criterion_weight': tensor([3., 1.]),  
'model__module_dropout': np.float64(0.2529847965068651),  
'model__optimizer': torch.optim.adamw.AdamW,  
'model__optimizer_lr': np.float64(0.0015304852121831463),  
'model__optimizer_weight_decay': np.float64(1.5339162591163613e-05)]
```

Project phase:

Scorecards



```
['total_rec_prncp',  
'last_fico_range_low', 'out_prncp',  
'installment', 'int_rate',  
'verification_status_Verified',  
'verification_status_Source Verified',  
'inq_last_6mths',  
'home_ownership_MORTGAGE',  
'inq_last_12m']
```

TABLE III
EXAMPLE SCORECARD ROW

Variable	Bin	Event Rate	IV	Points
total_rec_prncp	$(-\infty, 1996.59)$	0.713	0.691	-67.65

This row is an example of how the scorecard interprets one binned variable. For `total_rec_prncp`, loans with values below 1996.59 had an event rate of 0.713, meaning about 71.3% of loans in this group were default risk loans. Since this bin has a high event rate, it is considered a higher-risk group and receives negative scorecard points.

```
pd_nn_df["NN_risk_band"] = pd.cut(  
    pd_nn_df["PD_NN"],  
    bins=[0, 0.10, 0.25, 0.50, 0.75, 1.00],  
    labels=["Very Low", "Low", "Medium", "High", "Very High"],  
    include_lowest=True  
)
```

	actual	PD_NN	NN_risk_band
1881237	1	0.167996	Low
1422521	1	0.032815	Very Low
354508	1	0.067532	Very Low
885177	1	0.375518	Medium
1413526	1	0.105065	Low
1436786	0	0.521903	High
1687861	1	0.348972	Medium
1200057	0	0.517509	High
1044084	1	0.521933	High

Results of the scorecard

Test Report:

	precision	recall	f1-score
0	0.99	0.93	0.96
1	0.71	0.93	0.81
accuracy			0.93
macro avg	0.85	0.93	0.88
weighted avg	0.94	0.93	0.93

Test Confusion Matrix:

```
[[23386 1791]
 [ 335 4488]]
```

Project phase: Conclusion & future work



Limitations

Due to computational limits, feature engineering was applied before the full pipeline instead of inside a completely "honest" pipeline. This reduced model fitting time from about 20 minutes to 4 minutes, but it also means the workflow was not 100% leakage-safe(honest).



Future work

Future work should focus on predicting default risk at loan origination, since this is more practical for reducing loan losses before funding decisions are made. This would require careful removal of leakage variables that would not be available at the time of application.



Conclusion

Logistic Regression showed the strongest practical value because it had good performance while remaining interpretable enough to build a stakeholder-friendly scorecard.



Q&A

Any questions?
Ask away!

Give your audience space to
resolve doubts or concerns. Open
up the discussion before ending
the meeting.

